Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Reliability and Dependability Analysis of Complex Distributed Systems

Abstract

Modern distributed software—such as publish–subscribe middleware and microservices—heavily decouples components to achieve scalability. However, this decoupling obscures how cascading failures propagate across message brokers, topics, shared libraries, and host nodes. Detecting systemically critical components before deployment is challenging: operational telemetry does not yet exist, and traditional static code analysis cannot observe distributed communication topologies. When uncontained cascading failures reach production, they trigger costly downtime, emergency restart loops, and excessive energy consumption.

To solve this, we present **Software-as-a-Graph (SaG)**, an AI-driven framework for pre-deployment reliability analysis. SaG represents distributed system architectures as typed multigraphs across five core entities: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries. On this representation, SaG pairs two complementary techniques: **(1)** a relation-specific **Heterogeneous Graph Transformer (HGL)** that learns multi-layer propagation patterns to forecast cascading failure blast radii and rank critical components; and **(2)** an **Explainable Quality Attribution (RM)** baseline grounded in the ISO/IEC 25010 and ISO/IEC 25019 quality models that explains the structural root causes of vulnerability.

We evaluate SaG across 1,770 components from seven synthetic scenarios and three real-world open-source systems (Autoware.universe ROS 2, GCP Cloud Microservices, and the Train-Ticket benchmark) against independent discrete-event cascade simulators under a strict input–label independence guarantee. Our key findings show:

- **Superior Criticality Detection:** Typed graph learning outperforms standard structural metrics, achieving a +37.8% relative gain in identifying top-K critical components ($F_1@K = 0.467$ vs. 0.339) and strong out-of-distribution ranking ($\rho = 0.668$).
- **Value of Heterogeneous Modeling:** Heterogeneous message passing outperforms homogeneous graph neural networks by +0.197 in cross-scenario rank correlation, with Quality-of-Service (QoS) edge encoding providing critical robustness across differing architectures.
- **Actionable Explainability:** The quality attribution model successfully separates single points of failure (availability) from error propagation depth (fault tolerance), turning black-box graph predictions into concrete architectural fixes (e.g., broker replication vs. topic decoupling).
- **Real-World Generalization:** SaG transfers effectively to authentic cyber-physical and cloud-native systems, achieving high rank agreement ($\rho = 0.688$ to 0.778) and capturing up to 100% of failure-impactful services within the predicted critical set.

By enabling automated architectural analysis within CI/CD pipelines before deployment, SaG bridges the Architecture–Code Gap, fostering resilient, dependable, and energy-efficient software systems.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Publish–subscribe middleware, Distributed systems dependability, Cascading failures, Static system analysis, Architectural quality models

1. Introduction

1.1. Motivation

Modern large-scale distributed systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. From autonomous vehicles (ROS 2 [5]) and high-throughput enterprise backbones (Apache Kafka [4]) to cyber-physical systems (DDS [2]) and IoT meshes (MQTT [3]), pub-sub decouples producers and consumers in space, time, and synchronization [1]. Components communicate indirectly by sending and receiving messages through intermediate topics and brokers without maintaining direct references to one another. Furthermore, modern middleware lets developers specify deployment-time Quality-of-Service (QoS) policies—such as message durability, transport reliability, and delivery deadlines—to control how data flows under stress.

While this decoupling makes distributed systems scalable and flexible, it introduces a severe **visibility barrier** for system reliability and dependability:

- **Indirect Failure Pathways:** In traditional synchronous systems (such as REST or RPC architectures), component interactions follow explicit caller–callee call graphs. In pub-sub and event-driven meshes, there are no direct static references between publishers and subscribers. Cascading failures propagate across hidden logical pathways spanning brokers, shared topics, colocated execution nodes, and shared software libraries.

- **Distinct Failure Mechanisms:** Failures in distributed systems do not propagate in a single way. They manifest as either *sequential cascades* (e.g., slow consumer message-queue backlogs propagating downstream) or *simultaneous blast radii* (e.g., a shared library crash or node failure instantly disabling multiple colocated services at the same moment). Traditional architectural diagrams and static call graphs fail to capture these multi-layer interactions.

Fixing these vulnerabilities is easiest and cheapest **prior to deployment**, during architectural design and Continuous Integration / Continuous Delivery (CI/CD). However, pre-deployment is precisely when **no runtime telemetry, distributed tracing, or operational logs exist**. As a result, software architects and reliability engineers face two fundamental questions without runtime data:

1. *Which components and communication links are systemically critical to system reliability and availability?*
2. *Why are they critical, and what specific architectural fix (such as replicating a broker, decoupling a shared topic, or sandboxing a library) will most effectively eliminate that risk?*

Addressing this challenge is also critical for **system performance and sustainability**. When cascading failures occur in production, they trigger compute-intensive restart loops, failover storms, and retransmissions. Proactive, design-time architectural hardening prevents these failures before software is deployed, saving infrastructure costs and eliminating wasted energy.

1.2. Problem Statement: The Architecture–Code Gap

We formulate pre-deployment dependability analysis as two complementary tasks:

1. **Explainable Criticality Attribution:** Computing an interpretable, standards-grounded quality profile for every component—based on ISO/IEC 25010 [6] and ISO/IEC 25019 [7]—to explain *how* and *why* a component is vulnerable.
2. **Failure-Impact Forecasting:** Accurately predicting the global cascade blast radius and ranking the most failure-critical components using learned topological representations.

Existing software engineering approaches fail to bridge what we define as the **"Architecture–Code Gap"**: *a distributed system can have pristine, 100% bug-free source code within each service, yet remain fragile to catastrophic global outages due to hidden architectural single points of failure (SPOFs) or mismatched middleware QoS contracts.* Three prevailing paradigms leave this gap unaddressed:

- **Static Code Analysis (SCA):** Tools like SonarQube inspect source code complexity [8], modularity, and cohesion [9, 10] within single services. However, SCA cannot see the broader distributed network, message queues, or cross-host failure propagation.
- **Runtime Chaos Engineering:** Techniques like Chaos Monkey [11] and distributed tracing inject real faults into running staging or production environments. While effective at validating live systems, they require fully deployed infrastructure, carry operational risks, and arrive too late to guide initial architectural design.
- **Homogeneous Graph Centrality Metrics:** Standard network metrics (betweenness, PageRank, degree) [12, 13, 14, 15] flatten systems into simple, unweighted graphs. They treat all connections identically, failing to distinguish between a message topic, a shared library, and an execution host. Similarly, homogeneous Graph Neural Networks (GNNs) [16, 17, 18] ignore relation-specific message routing.

Currently, no unified framework combines typed multigraph modeling, source-code quality metrics, heterogeneous graph learning, and explainable quality attribution for pre-deployment analysis.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge this gap, we introduce **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Explainable Quality Attribution:** SaG combines code-level SCA metrics with topological graph properties into a hierarchical **Reliability**–**Maintainability (RM)** quality attribution model (Section 4). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), giving engineers actionable diagnostics.
4. **Heterogeneous Graph Learning for Failure Forecasting:** SaG deploys a **Heterogeneous Graph Transformer (HGL)** that uses relation-specific attention and message passing across typed entities to forecast cascading failure blast radii and rank critical components (Section 5).

To ensure methodological rigor, SaG enforces an **input–label independence guarantee**: the learned models and attribution baseline operate strictly on the derived analytical graph G_{analysis}, whereas ground-truth failure impacts are generated by independent discrete-event simulators executing over the raw structural topology $G_{\text{structural}}$ (Section 5.4).

Rationale for Graph Learning vs. Direct Simulation. Because discrete-event simulation $I^*(v)$ defines ground-truth criticality in this study, one might ask: *why train a graph neural network rather than running simulation sweeps directly?* If a complete simulator is available and compute budget is unlimited, simulation alone can identify critical components in an existing system. However, four practical reasons make our hybrid graph learning and attribution framework essential:

1. **Handling Unmeasured Components:** Discrete-event simulators only inject faults into active application processes, leaving passive infrastructure (such as message topics or host nodes) without direct simulation labels (30% to 47% of components per system). The learned GNN generalizes across both labeled and unmeasured entities.

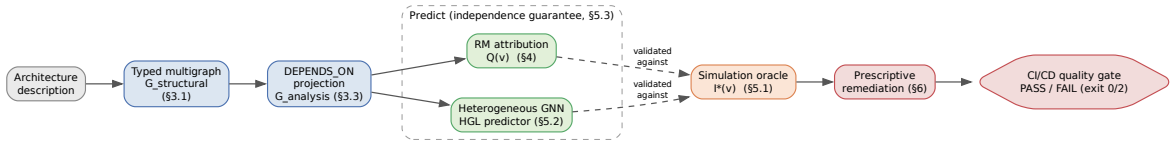

Figure 1: End-to-end architecture of the Software-as-a-Graph (SaG) framework: architecture manifest ingestion → typed multigraph → logical dependency projection → dual predictor paths (explainable RM baseline and Heterogeneous Graph Transformer) validated against independent discrete-event simulation.

2. **Variance Reduction & Stability:** Cascade simulations are noisy and highly sensitive to random seeds and propagation thresholds (label standard deviation across seeds reaches 0.416). The graph neural network learns a smooth, threshold-marginalized representation that stabilizes predictions across diverse operating regimes.
3. **Sub-Second Speed for CI/CD Quality Gates:** Running exhaustive multi-seed cascade simulations during every code commit or pull request is computationally slow (taking minutes or hours on large meshes). In contrast, trained graph transformers provide inference in under 50 ms, enabling instant CI/CD quality gating.
4. **Diagnostic Explainability:** Simulators provide only a single numerical impact score (how many services failed), but cannot explain *why*. Our ISO-grounded quality attribution model reveals the structural root causes (e.g., bottleneck articulation vs. high subscriber fan-out), directly guiding architects on how to refactor the system.

1.4. Research Questions

Our empirical evaluation investigates four key research questions:

RQ1 (Predictive Efficacy): *How accurately does graph learning predict cascading failure impact and identify critical components compared to traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) provide better failure predictions than homogeneous graph models?*

RQ3 (Impact of QoS and Model Sensitivity): *How do middleware Quality-of-Service (QoS) policies, quality weighting calibrations, and simulation thresholds impact prediction accuracy and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework generalize to real-world, open-source distributed systems across autonomous driving (ROS 2) and cloud-native microservice architectures?*

1.5. Key Contributions

This paper makes four principal contributions:

1. **A Formal Typed Architecture Model:** A multigraph representation of distributed pub-sub and microservice systems that derives logical dependencies and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures (Section 3).
2. **Heterogeneous Graph Learning for Dependability:** A relation-specific Heterogeneous Graph Transformer (HGL) with 16-dimensional QoS edge feature encoding tailored for pre-deployment failure blast-radius prediction (Section 5).
3. **Standards-Grounded Explainable Attribution:** An interpretable Reliability–Maintainability (RM) quality baseline grounded in ISO/IEC 25010/25019 that bridges code-level SCA metrics with system-level topological criticality (Section 4).

4. **Empirical Evaluation & Real-World Validation:** A rigorous empirical evaluation across seven synthetic topologies (1,545 components) and three authentic real-world systems (Autoware.universe ROS 2, GCP Cloud Microservices, and Train-Ticket; 225 components) under strict input–label independence, establishing the exact conditions where typed graph learning delivers decisive advantages (Sections 6–7).

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work in distributed systems dependability, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model and dependency projection rules. Section 4 presents the interpretable RM attribution baseline. Section 5 details the Heterogeneous Graph Transformer architecture and simulation ground truth. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ4. Section 8 discusses architectural implications, sustainability impacts, threats to validity, and concluding remarks.

2. Related Work

This research intersects four major domains: distributed systems dependability, static system analysis, graph representation learning for network vulnerability, and software quality models.

2.1. Dependability in Distributed and Pub-Sub Systems

The publish–subscribe paradigm provides foundational communication decoupling for distributed systems [1]. Formal middleware specifications—such as DDS [2], MQTT [3], ROS 2 [5], and distributed commit logs like Kafka [4]—enable fine-grained QoS policies governing durability, liveliness, and transport reliability. Prior research on pub-sub dependability has predominantly focused on runtime mechanisms: fault-tolerant event routing, broker clustering, dynamic consensus, and adaptive message retransmission.

In parallel, runtime verification and chaos engineering [11] systematically inject faults into staging or production clusters to observe steady-state recovery. While essential for validating operational resilience, runtime techniques operate late in the software lifecycle, require complete infrastructure deployments, and cannot evaluate architectural alternatives during initial design. Our work addresses the complementary, pre-deployment phase: predicting cascading vulnerability from architectural models *before* systems are deployed.

2.2. Static Code Analysis vs. Static System Analysis

Traditional Static Code Analysis (SCA) analyzes source code abstract syntax trees (ASTs) to measure cyclomatic complexity [8], module coupling (e.g., LCOM, CBO) [9, 10], and code duplication. While SCA effectively identifies internal code smells and defect-prone modules [19, 20, 21, 22], it is entirely oblivious to inter-process communication, container placement, and middleware topologies.

To bridge this "Architecture–Code Gap," **Static System Analysis (SSA)** elevates static analysis to the system architecture level. By modeling distributed components, message channels, and infrastructure hosts as a global dependency graph, SSA frameworks propagate code-level quality attributes across topological links. This enables continuous pre-deployment verification in modern CI/CD pipelines [23, 24], catching structural anti-patterns [25, 26, 27, 28] and architectural degradation [29, 30] at commit time.

2.3. Network Science and Graph Representation Learning

Network science offers established metrics for identifying critical network elements, including degree, closeness, betweenness centrality [12, 14], articulation points, and PageRank [13, 15]. Studies on random failure tolerance [31], cascading overload [32], and interdependent networks [33] have provided deep mathematical foundations for systemic vulnerability. However, standard network metrics suffer from two fundamental limitations when applied to software architectures:

1. *Dimensional Collapse:* A single centrality score cannot distinguish a structural Single Point of Failure (SPOF) from an error-propagating cascade hub or a high-maintenance bottleneck.
2. *Semantic Collapse:* Discarding node and edge types treats an asynchronous pub-sub topic the same as a physical host or a shared library, masking distinct failure propagation mechanics.

To overcome the limitations of hand-engineered graph metrics, recent research has applied machine learning on graphs for network dismantling and criticality estimation (e.g., FINDER [34], DrBC [35], and PowerGraph [36]). However, most existing models rely on homogeneous message passing (GCN [16], GraphSAGE [17], GAT [18]). Because distributed middleware is inherently multi-typed and heterogeneous, homogeneous aggregation mixes fundamentally incompatible semantic relationships. Heterogeneous Graph Neural Networks (RGCN [37], HAN [38], HGT [39], MAGNN [40]) address this by parameterizing relation-specific transformations. We build upon the Heterogeneous Graph Transformer (HGT) architecture [39] to maintain typed relational semantics during cascade impact forecasting.

2.4. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the ISO/IEC 25010:2023 product quality model [6] (comprising nine characteristics including Reliability, Maintainability, and Security) and the ISO/IEC 25019:2023 Quality-in-Use standard [7] (evaluating stakeholder harm over Beneficialness, Freedom from Risk, and Acceptability). Quality evaluation distinguishes between *internal quality* (measured statically on software artifacts at rest) and *external quality* (measured dynamically on executing systems) [41, 42].

Synthesizing multi-attribute structural metrics into actionable quality indices constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The Analytic Hierarchy Process (AHP) [43] provides a mathematically rigorous pairwise-comparison framework equipped with an explicit Consistency Ratio ($CR \leq 0.10$) to validate expert weighting schemes. In this work, we use AHP to construct an audited, interpretable Reliability–Maintainability attribution baseline that serves as a transparent benchmark for our learned GNN models.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), and the dual graph views utilized throughout the framework (Section 3.3).

3.1. Formal Multigraph Definition

We model a complex distributed system as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and coupling strength.

Application and Library entities additionally ingest static code metrics computed via SCA tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), directly bridging code-level fragility with topological analysis.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized environment	Bare-metal server, Kubernetes worker, Cloud V
Library (V_{lib})	Shared software package or driver dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links exhibit varying degrees of coupling based on their Quality-of-Service (QoS) contracts. For example, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services far more tightly than a `BEST_EFFORT` telemetry stream.

For each structural communication edge e, we compute an edge coupling weight $w_E(e) \in [0, 1]$ derived from the QoS profile of the mediating topic:

$$w_E(e) = 0.85 \cdot (w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}) + 0.15 \cdot q_{\text{payload}} \tag{3}$$

where $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport priority scores, with AHP weights $(0.30, 0.40, 0.30)$ ($CR < 0.05$). Durability is assigned the highest weight because it dictates message persistence across network partitions. A minimum floor $w_E(e) \geq 0.01$ ensures that best-effort edges remain visible to graph traversals.

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"):

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	**Dependency Category**	**Structural Pattern (Dependent → Dependency)**	**Derived Weight** (w)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$\max_t w_E(t)$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$\max_t w_E(t)$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	`app_to_lib`	Application → Shared Library it `USES`	$w_V(\text{app})$
6	`broker_to_broker`	Broker ↔ Broker (bidirectional, colocated on same host)	$w_V(\text{node})$

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A key insight of the SaG model is distinguishing between two fundamentally different failure modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers experience data starvation. The failure propagates step-by-step through topics and message queues.

- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single event.

Retaining entity types and relation-specific dependency rules allows SaG to model both mechanisms, whereas untyped graphs collapse them into identical edges.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw graph containing physical and structural edges (`PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, `USES`). This view is consumed exclusively by discrete-event simulators to execute unbiased failure injections (Section 5.1).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), allowing architects to evaluate criticality at subsystem levels (e.g., following MIL-STD-498 hierarchical structures).

4. Interpretable Attribution as a Quality Baseline

To provide actionable architectural explanations alongside data-driven predictive rankings, SaG decomposes component and relationship criticality into a multi-dimensional, standards-grounded quality attribution profile.

4.1. Grounding in ISO/IEC Standards

Following **ISO/IEC 25010:2023** (Product Quality Model) [6] and **ISO/IEC 25019:2023** (Quality-in-Use) [7], we formulate two core formal criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system's capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated primarily across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics
Reliability (R)	**Fault Tolerance (FT)**	How broadly does failure propagate?	Reverse PageRank on $G^{\top}$, in-deg
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-

Coverage Scope: SaG focuses on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, e.g., ISO 26262 ASIL ratings) and Security (which requires explicit threat models, e.g., STRIDE) are declared external to purely structural analysis and are left for domain-specific extensions.

4.2. Typed Node Feature Representation

SaG extracts rich feature vectors tailored to the 5 distinct entity types in the software multigraph:

- **Application ($|V_{\mathbf{app}}|$, 23 dims):** Indices 0–17 represent shared topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load). Indices 18–22 capture source code metrics extracted via Static Code Analysis (SCA): Lines of Code (LOC), Cyclomatic Complexity, Martin's Instability metric ($I_{\text{code}} = \frac{C_e}{C_a+C_e}$), Lack of Cohesion in Methods (LCOM), and composite Code Quality Penalty (CQP).
- **Library ($|V_{\mathbf{lib}}|$, 25 dims):** Same shared topological (0–17) and code quality (18–22) metrics as Application, plus two library-specific extras (indices 23–24): the size of the transitive reverse-`USES` closure (normalized) and the count of distinct subscribers reachable from that closure's published topics (normalized) — the two structural drivers of a library's blast radius under both simulators' cascade rules that the code-quality metrics alone do not capture.
- **Broker ($|V_{\mathbf{broker}}|$, 19 dims):** Indices 0–17 shared topological metrics; index 18 represents normalized queue buffer capacity.
- **Topic ($|V_{\mathbf{topic}}|$, 22 dims):** Indices 0–17 shared topological metrics; indices 18–21 capture publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node ($|V_{\mathbf{node}}|$, 20 dims):** Indices 0–17 shared topological metrics; indices 18–19 capture normalized CPU core allocation and physical memory (RAM).

4.3. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. The quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [43]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^{\top}_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{4}$$

   where RPR is Reverse PageRank and $\text{CDPot}_{\text{enh}}$ is an enhanced Cascade Depth Potential term.
2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component's own QoS weight:

$$A(v) = 0.35 \cdot \text{AP}_c^{\text{dir}}(v) + 0.25 \cdot \text{QSPOF}(v) + 0.25 \cdot \text{BR}(v) + 0.10 \cdot \text{CDI}(v) + 0.05 \cdot w(v) \tag{5}$$

3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability hierarchically:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v), \quad \alpha = 0.36 \tag{6}$$

   Intra-dimension pairwise comparison matrices are audited against Saaty's consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights are a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports the sensitivity of ranking accuracy to λ).
4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \tag{7}$$

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \tag{8}$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \tag{9}$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This enables actionable diagnostics: a service scoring high on A but low on FT is diagnosed as a pure SPOF requiring horizontal replication, whereas a service scoring high on FT is an error cascade hub requiring circuit breakers and bulkhead isolation.

5. Graph Learning for Failure-Impact Prediction

While the interpretable RM baseline provides explainable quality profiles, complex non-linear failure interactions and non-local cascade propagations benefit significantly from graph representation learning. This section details our Heterogeneous Graph Transformer (HGT) architecture, 16-dimensional edge representation, multi-task prediction heads, dimension-masked loss formulation, ground-truth simulation oracles, and the strict input–label independence guarantee.

5.1. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal three-oracle taxonomy:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via `FaultInjector`, this oracle injects an unexpected node crash at component $v \in V$, propagates cascading failures across dependent topics, brokers, and network links using breadth-first dynamic traversal, and computes the fraction of surviving subscriber feeds severed by the outage. $I^*(v) \in [0, 1]$ serves as the primary continuous target label for training and evaluating GNN predictors.
- **Multi-Metric Composite Oracle ($I_{\textbf{comp}}(v)$):** Implemented via `FailureSimulator`, this oracle evaluates a multi-faceted failure impact vector:

  $$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{10}$$

  $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.
- **Dynamic Queue-Flow Oracle ($I_{\textbf{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy discrete-event simulation framework [44], this oracle models message emission rates, stochastic network latencies, broker buffer saturation, and dropped delivery counts under fault injection.
- **Relationship (Edge) Removal Oracle ($I_{\textbf{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping both endpoint components alive:

  $$I_{\text{edge}}(u, v) = I_{\text{comp}}(G \setminus \{(u, v)\}) - I_{\text{comp}}(G) \tag{11}$$

These oracles are not interchangeable, and their disagreement bounds what any single one can support. Measured across seven scenarios and five seeds, mean Spearman agreement is $\rho = 0.765$ for (I_{dyn}, I^*), $\rho = 0.465$ for $(I_{\text{comp}}, I_{\text{dyn}})$, and $\rho = 0.397$ for (I_{comp}, I^*), with per-scenario minima of 0.550, 0.121, and 0.097 respectively. Agreement on the *critical set* is weaker still than agreement on ordering: mean top-K Jaccard overlap is 0.26–0.31 for every pair, including the strongest. A result established against one oracle is therefore not evidence for a claim measured against another, and the instruction to "simply simulate" is under-specified — it does not say which simulator, at which propagation threshold, under which seed, and the answer materially changes the critical set. We report this disagreement as a bound on our own construct validity rather than as corroboration.

5.2. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `RUNS_ON`, `CONNECTS_TO`, `USES`, etc.), we implement a 3-layer **Heterogeneous Graph Transformer (HGT)** architecture [39] with hidden dimension $D = 64$ and 4 attention heads.

5.2.1. Edge Feature Encoding (16-Dimensional)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$:

- **Index 0:** Scalar QoS weight $w(e) \in (0, 1]$, computed via the harmonic mean of reliability, durability, priority, deadline, and blocking multipliers.
- **Index 1:** Normalized path count through edge e in G_{analysis}.
- **Indices 2–8:** 7-bit one-hot encoding for edge relationship types (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`).
- **Indices 9–15:** 7 explicit QoS profile dimensions, non-zero only for `PUBLISHES_TO`/`SUBSCRIBES_TO` edges (all other edge types receive zeros): (9) Reliability policy score (0.0 = best-effort, 1.0 = reliable); (10) Durability policy score (0.0 = volatile, 0.5 = transient local, 0.6 = transient, 1.0 = persistent); (11) Normalized message priority (0.0/0.33/0.66/1.0 for low/medium/high/urgent); (12) Deadline constraint active flag (0/1); (13) Log deadline $\log_{10}(1 + \text{deadline\_ns}/10^6)$; (14) Log max blocking time $\log_{10}(1 + \text{max\_blocking\_ms})$; and (15) a QoS heterogeneity flag (1 if the edge's reliability/durability/priority triple deviates from the scenario's modal QoS profile, 0 otherwise).

The `EdgeFeatureEncoder` projects e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention, the edge projection is added directly to the destination node representation: $\tilde{h}_v = h_v + e'_{uv}$.

5.2.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v with meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (dimensions 19–23 depending on $\tau(v)$) are mapped into D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)} x_v)) \tag{12}$$

2. **Relational Mutual Attention:** Query, Key, and Value projections compute relation-specific attention across heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u) W_{\text{att},\phi(e)} Q(\tilde{v})^\top}{\sqrt{D/H}} \right) \tag{13}$$

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{14}$$

3. **Bidirectional Message Passing:** To capture downstream backpressure and upstream cascading failures, message passing is executed over both forward and reverse relations (G_{analysis} and $G_{\text{analysis}}^{\top}$).
4. **Residual Aggregation and Layer Normalization:**

$$h_v^{(l)} = \text{LayerNorm}\left(h_v^{(l-1)} + \text{Dropout}\left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v)\right)\right) \tag{15}$$

5.3. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG branches into specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

5.3.1. Dimension-Masked Loss Formulation

The joint optimization objective balances regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.1 \cdot \mathcal{L}_{\text{consistency}} + 0.3 \cdot \mathcal{L}_{\text{edge}} \tag{16}$$

where $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is ListMLE ranking loss [45], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}(\hat{I}^*(v), 0.8\hat{R}(v) + 0.2\hat{M}(v))$.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than static code maintainability, maintainability ground truth is unobserved during dynamic simulation. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{17}$$

This prevents the unmeasured maintainability head from being penalized or driven toward zero by backpropagation.

5.3.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^{\top}$), the composite score is evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{18}$$

where $M_{\text{static}}(v)$ is drawn directly from the structural analyzer's maintainability baseline (y_{rm} maintainability column), combining learned dynamic reliability with static source-code maintainability. The implementation deliberately refuses to emit $Q_{\text{domain}}(v)$ unless the checkpoint's maintainability head actually received real supervision — under the sole labeler used in every experiment reported here (`FaultInjector`, $m = [1, 0]$), maintainability is unmeasured, so $Q_{\text{domain}}(v)$ is never populated in these results. Domain reweighting sensitivity is instead evaluated directly against the static RM baseline, reported as a dedicated ablation (Section 7.3).

5.4. Input–Label Independence Guarantee

To guarantee experimental rigor and eliminate data leakage, SaG enforces strict architectural decoupling:

- **Feature Space:** Derived exclusively from G_{analysis} (using static structural topology, SCA metrics, and declared QoS contracts).
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs are ever exposed as input features to the GNN or attribution baseline.

6. Experimental Setup

6.1. Datasets and System Corpus

Our evaluation corpus comprises 1,770 components across ten distributed system architectures:

Table 4: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$\|V\|$	$\|V_{\text{app}}\|$	Topics	Brokers	Hosts	Libs	$\|E\|$
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [46]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [47]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [48]	Real-World Microservices	90	41	30	3	8	8	162
Total		**1,770**						

$|V|$ is the sum of all five entity-type counts per scenario and sums to exactly the corpus total stated above. $|E|$ counts every raw structural relationship instance recorded in the scenario file (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`) — the substrate the simulation oracles traverse — not the denser derived `DEPENDS_ON` projection used for GNN training.

The seven synthetic scenarios are generated using statistical topology generators spanning diverse degree distributions, clustering coefficients, and middleware QoS configurations. The three real-world architectures are hand-transcribed from open-source repositories using dedicated architectural adapters.

6.2. Baselines and Evaluated Predictors

We compare five primary predictor configurations:

1. **Topo-BL:** Unweighted structural centrality (betweenness centrality and articulation point scoring).
2. **Topo-QoS:** QoS-weighted topological centrality baseline.
3. **RM / $Q(v)$:** Deterministic hierarchical quality attribution baseline (Section 4).
4. **GL / GL-QoS:** Homogeneous Graph Attention Networks (GAT) [18] trained on the flattened, untyped graph projection.
5. **HGL / HGL-QoS:** Relation-specific Heterogeneous Graph Transformers (Section 5).

6.3. Evaluation Metrics and Protocols

- **Ranking Accuracy:** Spearman rank correlation (ρ) and Kendall's tau (τ) between predicted rankings and ground-truth simulated impact $I^*(v)$.
- **Critical-Set Identification:** $F_1@K$, Precision@K, and Recall@K for top-K critical components, where $K = \lceil 0.20 \cdot |V_{\text{app}}| \rceil$.
- **Statistical Significance:** Paired Wilcoxon signed-rank tests [49] ($p < 0.05$) and bootstrap 95% confidence intervals ($B = 2,000$) [50, 51].

6.3.1. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits pinned by node identity within each scenario, evaluated over five random seeds $\{42, 123, 456, 789, 2024\}$.
- **Inductive Leave-One-Scenario-Out (LOSO):** Across all eight cached scenarios (the seven synthetic scenarios plus the ATM case study, Section 7.4), models are trained on the remaining seven and evaluated zero-shot on the held-out scenario, for eight folds total, to test out-of-distribution generalizability.
- **Real-World Architectural Transfer:** Evaluating zero-shot transfer on authentic open-source architectures.

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 5 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

Table 5: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$ (seed means over 5 seeds with bootstrap 95% CIs; n is held-out Application count).

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.283	0.701	**0.790**	0.689	0.789	0.649
Enterprise	60	0.420	0.789	0.875	0.459	0.880	**0.891**
Financial Trading	12	0.289	0.700	0.672	0.536	**0.854**	0.770
Healthcare	10	−0.101	**0.772**	0.590	0.463	0.652	0.645
Hub-and-Spoke	14	0.234	0.359	**0.554**	0.052	0.534	0.296
IoT Smart City	40	−0.063	0.073	0.238	0.259	**0.881**	0.842
Microservices	18	0.401	**0.707**	0.564	0.571	0.483	0.476
Mean	—	**0.209**	**0.586**	**0.612**	**0.433**	**0.725**	**0.653**

Table 6: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGL vs. Topo-BL	**+0.516**	7/7	0.0	**0.0156**	**Statistically Significant** ($p < 0.05$)
HGL vs. GL-QoS	**+0.292**	6/7	1.0	**0.0312**	**Statistically Significant** ($p < 0.05$)
HGL vs. Topo-QoS	**+0.139**	5/7	9.0	0.469	Not significant
GL vs. Topo-QoS	**+0.026**	4/7	11.0	0.688	Not significant
HGL vs. GL	**+0.113**	4/7	9.0	0.469	Not significant
HGL-QoS vs. HGL	**−0.072**	1/7	3.0	0.078	Not significant (marginal; HGL-QoS trails in-distribution)

Table 7: Inductive Leave-One-Scenario-Out (LOSO) evaluation.

Model Variant	Mean LOSO ρ	Std ρ	Critical-Set $F_1@K$	Requires Training
Topo-BL	0.109	0.150	0.222	No
Topo-QoS	0.522	0.285	0.339	No
RM / $Q(v)$ Baseline	−0.014	0.147	0.237	No
GL (Homogeneous)	0.381	0.164	0.436	Yes
GL-QoS (Homogeneous)	0.501	0.167	0.442	Yes
HGL (Typed Heterogeneous)	0.578	**0.116**	**0.467**	Yes
HGL-QoS (Typed + QoS)	**0.668**	0.096	0.457	Yes

7.1.1. Out-of-Distribution (LOSO) Generalization

Under inductive Leave-One-Scenario-Out (LOSO) cross-validation, the model must predict cascading criticality on an unseen system topology:
Eight LOSO folds are reported (the seven synthetic scenarios plus the ATM case study of Section 7.4), each holding out one scenario and training on the remaining seven.

Key Insights for RQ1:

1. **Critical-Set Detection Advantage:** HGL achieves a 37.8% relative improvement in critical-set identification ($F_1@K = 0.467$ vs. 0.339 for Topo-QoS).
2. **Out-of-Distribution Superiority:** Under LOSO validation, the QoS-aware typed model (HGL-QoS) is the best-performing configuration overall ($\rho = 0.668$ vs. 0.521 for Topo-QoS), substantially outperforming non-learning baselines and homogeneous GNNs ($\rho = 0.381$–0.501). The RM baseline is essentially non-predictive under distribution shift ($\rho \approx -0.01$), confirming that RM's value lies in interpretable attribution rather than standalone ranking.
3. **In-Distribution Scope Condition:** In-distribution, both HGL ($\rho = 0.725$) and GL ($\rho = 0.612$) outperform structural baselines on point estimates, but their margin over Topo-QoS is constrained by topology variance. Notably, in-distribution the QoS-aware variant (HGL-QoS, $\rho = 0.653$) trails the base typed model rather than matching it — see Section 7.3's QoS Feature Encoding discussion for the full trade-off.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the contribution of node and edge typing, we compare relation-specific HGL against homogeneous GL across different validation regimes:

- **In-Distribution:** Heterogeneous HGL leads homogeneous GL by $\Delta\rho = +0.113$ (0.725 vs. 0.612; not statistically significant, $p = 0.469$, 4/7 scenarios won). On familiar topologies, homogeneous GNNs can partially approximate type distinctions through structural degree signatures, but the typed model still holds a consistent point-estimate edge.
- **Out-of-Distribution (LOSO):** Heterogeneous HGL outperforms homogeneous GL by +0.197 ($\rho =$ 0.578 vs. 0.381) and exhibits substantially lower variance across folds ($\sigma = 0.116$ vs. 0.164). When encountering unseen topologies, relation-specific message passing is essential for generalizing failure mechanics — and the gap widens further to +0.167 comparing the QoS-aware variants (HGL-QoS 0.668 vs. GL-QoS 0.501), the best-performing configuration in either family.

7.2.1. Empirical Edge-Removal Analysis

We further probed edge criticality by simulating the removal of candidate relationship channels while keeping both endpoint components alive. On the `av_system` topology across 50 candidate bridge edges, 46 edges exhibited zero downstream cascade impact, while the 4 edges with non-zero impact were direct library communication channels (`PUBLISHES_TO` / `SUBSCRIBES_TO`). This demonstrates that individual communication links are largely redundant, whereas component-level failures and shared library dependencies induce disproportionate systemic disruption.

7.3. RQ3: Ablations and Sensitivity Analysis

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes in the GNN (HGL-QoS) does not yield a uniform effect — it trades in-distribution accuracy for out-of-distribution generalization. In-distribution, HGL-QoS *trails* the base typed model ($\rho = 0.653$ vs. 0.725; $\Delta\rho = -0.072$, marginal at $p = 0.078$, HGL-QoS wins only 1/7 scenarios), consistent with QoS features adding redundant signal (and mild overfitting risk) on topologies the model has already seen, since the derived `DEPENDS_ON` graph already embeds much of the same routing and coupling information. Under LOSO, the relationship reverses sharply: HGL-QoS is the single best-performing configuration in the entire study ($\rho = 0.668$ vs. 0.578 for HGL; $\Delta\rho = +0.090$). When the topology itself is unseen, the explicit QoS signal generalizes in a way the structural encoding alone does not. We read this as evidence that QoS encoding is not "redundant" so much as *situational*: valuable insurance against distribution shift, at a small in-distribution cost.

7.3.2. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline by sweeping a shrinkage parameter $\lambda \in [0, 1]$ blending internal term weights from uniform prior ($\lambda = 0$) to calibrated AHP judgement ($\lambda = 1$):

Table 8: Sensitivity of RM rank correlation to AHP shrinkage parameter λ.

λ **Setting**	**0.00 (Uniform)**	**0.50**	**0.70 (Default)**	**0.80**	**1.00 (Raw AHP)**
Mean Rank Correlation (ρ)	-0.0514	-0.0256	-0.0190	-0.0142	-0.0069

As illustrated in Figure 4, the calibrated AHP judgement ($\lambda = 1.00$) is monotonically less anti-correlated with ground truth than uniform weighting, improving by $+0.045\,\rho$ across the sweep and validating the internal consistency of the expert hierarchy. We note explicitly that every λ setting remains negative on this population: the RM baseline is weakly *anti*-correlated with $I^*(v)$ across the whole shrinkage range, consistent with its near-zero-to-negative LOSO performance (Table 7). AHP calibration makes the RM baseline less wrong, not accurate; its primary role remains explainable attribution rather than standalone ranking.

7.3.3. Convergent Validity Across Simulation Oracles

To test construct validity, we compared node criticality rankings across our three simulation engines: the topological cascade reachability oracle $I^*(v)$ (`FaultInjector`), the multi-metric composite oracle $I_{\text{comp}}(v)$ (`FailureSimulator`), and the discrete-event queue-flow simulator $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`).

Across all seven synthetic scenarios, the behavioural SimPy simulator (I_{dyn}) and the topological cascade reachability oracle (I^*) exhibited strong rank convergence:

- $I_{\textbf{dyn}}$ **vs.** $I^*(v)$**:** Mean Spearman $\rho = \mathbf{0.7647}$ (ranging from 0.5497 to 0.9378), with mean Kendall $\tau = 0.633$ and top-K Jaccard overlap of 0.312.
- $I_{\textbf{comp}}$ **vs.** $I_{\textbf{dyn}}$**:** Mean Spearman $\rho = 0.4646$ (ranging from 0.1208 to 0.6575).
- $I_{\textbf{comp}}$ **vs.** $I^*(v)$**:** Mean Spearman $\rho = 0.3970$ (ranging from 0.0974 to 0.5778).

This strong agreement between independent behavioural queue simulations and structural cascade reachability confirms that our continuous target label $I^*(v)$ captures genuine dynamic service disruption.

7.3.4. Domain-Specific Weighting Sensitivity

We investigated the impact of Context of Use reweighting $\vec{\omega} = [q_R, q_M]^\top$ across 10 evaluation scenarios by sweeping the composite reliability weight w_R over its full range and reading off three points on that one curve: the shipped static default ($w_R = 0.80$), an unweighted equal split ($w_R = 0.50$), and the domain-derived value per scenario. Domain-derived weighting improves mean rank correlation by $\Delta\rho = +0.0264$ over the static default, but *underperforms* equal weighting by $\Delta\rho = -0.1097$ — domain derivation is not, on this evidence,

a ranking-accuracy improvement over the simplest possible baseline. The reason is structural rather than a modelling failure: across all six declared domains, the domain-derived w_R sits within 0.04 of the static 0.80 default (range $[0.70, 0.76]$), so mean Kendall rank fidelity between domain-derived and static rankings is $\tau = 0.9677$ — the two weightings rank components almost identically, and the free parameter Context of Use actually moves is too small for any weighting confined to it to move ρ substantially. We therefore reframe this ablation's conclusion: operational Context of Use reweighting is an *attributional* mechanism — explaining why a component matters in stakeholder terms — not a ranking-improvement device, consistent with the scoping in Section 4.1.

7.3.5. Threshold and Normalization Sensitivity

We swept 7 scenarios × a cascade propagation-threshold parameter $\in \{0.0, 0.1, 0.2, 0.35, 0.5, 0.75, 1.0\}$ controlling the ground-truth oracle's eligibility cutoff for cascade propagation, and separately swept 3 label-normalization techniques (min-max, standard, rank-quantile) holding the threshold fixed. Ranking performance is more sensitive to the propagation threshold ($\Delta\rho = 0.1896$ across the extreme settings; mean ρ ranges from -0.173 at threshold 0 to $+0.017$ at threshold ≥ 0.35, plateauing thereafter) than to normalization choice ($\Delta\rho = 0.0158$ across the three schemes). A small spread under either sweep means the reported ordering is stable under that free parameter of the ground truth and scorer — it is not, by itself, evidence that the ordering is *correct*.

7.3.6. Anti-Pattern Detection and CI/CD Quality Gates

Validating our rule-based architectural anti-pattern catalog against the multi-metric composite oracle $I_{\text{comp}}(v)$ yielded a mean detection $F_1 = 0.3781$ across 8 system scenarios — but F_1 alone understates what the catalog does: mean precision is 0.239 against mean recall 0.900, meaning the catalog flags 94.2% of all components on average and trades precision for near-exhaustive coverage of true positives. This is a deliberate high-recall CI/CD posture (a missed critical component is more costly than a false positive requiring manual triage) rather than an accuracy shortfall, but should be read as such rather than via F_1 in isolation. One detector, `DEEP_PIPELINE`, is excluded from this evaluation: it enumerates every simple source-to-sink path and does not terminate within ten minutes on the 50-application Healthcare topology — the blowup is itself a reportable scalability limit of exhaustive path enumeration, not a measurement gap. Analysis execution times for the remaining 18 detectors ranged from 0.04 s (small scenarios, 50 nodes) to 54.85 s (enterprise event meshes, 300 nodes), confirming that SaG runs comfortably within standard pre-commit and pull-request CI/CD quality gate budgets.

We additionally stratify the RM composite's rank correlation by node type: $\rho = 0.503$ (Application), 0.395 (Broker), 0.142 (Node), while the *pooled* correlation across all types is $\rho = 0.028$ — outside the per-type range entirely. This is a Simpson's-paradox effect (aggregating across populations with different scales/base rates reverses the within-group trend), and it means the pooled figure is not a summary of the per-type figures: only the stratified, per-type numbers should be quoted when discussing RM's structural predictive power.

7.3.7. HGT Attention Weight Analysis

Extracting relational attention matrices from the trained HGT model on the ATM Case Study (Figure 3) revealed that the model places its highest single attention weight ($\alpha_{uv} = 1.00$) on a `USES` edge (an Application's dependency on a shared Library), with the next tier ($\alpha_{uv} \approx 0.50$) split between a `ROUTES` edge (Broker → Topic) and further `USES`/`SUBSCRIBES_TO` edges. This pattern — dominant attention on library-dependency and broker-routing channels rather than direct publish/subscribe message flow — reflects the shared-library blast-radius and broker-centrality failure pathways identified elsewhere in this study (Section 7.2's edge-removal analysis, Section 3's simultaneous-failure semantics for shared libraries) rather than sequential pub-sub cascade propagation.

7.4. RQ4: Real-World Distributed Software Architecture Validation

To assess external validity on authentic production architectures, we evaluated SaG on three open-source distributed systems.

Table 9: Empirical validation on authentic real-world distributed software architectures.

Real-World Architecture	$\|V\|$	$\|V_{\mathbf{app}}\|$	Spearman ρ	Kendall τ	$F_1@K$	Tie-Robust F_1	Non-Zero	Gain
Autoware.universe (ROS 2)	75	32	**0.688 ± 0.009**	0.517	**0.800**	0.800	19 / 32	+0.36
Cloud Microservices Mesh	60	22	**0.778 ± 0.001**	0.639	**1.000**	0.760	8 / 22	+0.01
Train-Ticket Booking Mesh	90	41	**0.759 ± 0.001**	0.605	**1.000**	0.810	14 / 41	+0.26

Key Insights for RQ4:

1. **Strong Real-World Rank Agreement:** SaG achieves high rank correlation on Cloud Microservices ($\rho = 0.778$) and Train-Ticket ($\rho = 0.759$), and solid agreement on Autoware.universe ($\rho = 0.688$).
2. **Critical-Set Containment:** Every single application with non-zero cascading impact in Cloud Microservices and Train-Ticket is successfully captured within the predicted top-K critical set (tie-robust $F_1@K = 0.760$ and 0.810).
3. **Substantial Predictive Gain:** SaG outperforms raw degree centrality by +0.360 on Autoware and +0.264 on Train-Ticket, demonstrating that typed dependency derivation captures critical architectural semantics beyond superficial connectivity.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Our empirical findings provide clear guidance for software architects and reliability engineers:

- **When to Use Graph Learning:** Heterogeneous GNNs provide the greatest return when the objective is identifying the critical top-K shortlist of components for architectural hardening ($F_1@K = 0.467$, HGL), and when evaluating new, unseen system architectures out of distribution ($\rho = 0.668$, HGL-QoS). The choice between the base typed model and its QoS-aware variant is itself a design decision: HGL for in-distribution accuracy, HGL-QoS when the deployment target is expected to differ structurally from the training corpus (Section 7.3).

- **When to Use Interpretable Attribution:** For root-cause diagnosis and remediation planning, the deterministic RM profile is indispensable. Distinguishing whether a service is critical due to single-point-of-failure exposure (Availability) or wide error propagation (Fault Tolerance) directly dictates whether to introduce load-balanced replicas or circuit-breaker policies.

8.2. Threats to Validity

- **Construct Validity:** Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To characterise construct divergence we evaluated three independent simulation engines (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`). Their agreement is partial and we report it as a bound rather than a mitigation: mean $\rho = 0.765$ between dynamic queue-flow simulation and cascade reachability, but only $\rho = 0.397$ between the composite and cascade oracles, with top-K Jaccard overlap of 0.26–0.31 for every pair (Section 5.1). A further 30–47% of components per scenario carry no simulated ground truth at all, because the cascade model cannot express direct Topic or Node failure. Accordingly, results established against one oracle should not be read as claims about another, and none of them are claims about observed production outages: the reported correlations measure surrogate fidelity to a simulator, not predictive accuracy against deployed systems.

- **Internal Validity:** Potential feature leakage is prevented by strict graph view separation: predictors operate on G_{analysis}, whereas ground-truth simulators operate on $G_{\text{structural}}$. We additionally identified and fixed a normalization defect in the code-quality scoring pipeline: a min-max helper previously assigned the maximal Code Quality Penalty to any zero-variance population, which is correct for

one genuine node but was indistinguishable from "many nodes carrying no code-quality data at all" — the exact shape of every Library in our three real-world scenarios, which carry no source-level `code_metrics`. The fix (scoring an undifferentiated multi-node population as zero rather than maximal penalty) changes Library Maintainability tier classifications in the real-world corpus but, as expected for a fix confined to a population with no variance to rank, leaves Application-population rank correlation against $I^*(v)$ effectively unchanged (Table 9: $\Delta\rho \in \{-0.003, 0.000, 0.000\}$ across the three real-world scenarios).

- **External Validity:** While our corpus spans ten distinct system domains and three authentic open-source architectures (ROS 2 and microservices), future work should evaluate larger enterprise deployments with hundreds of microservices.

- **Conclusion Validity:** Given the heavy-tailed, non-normal distribution of cascading failure impacts, all statistical comparisons utilize non-parametric rank correlation (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. We note one aggregation hazard directly: the RM composite's rank correlation pooled across node types ($\rho = 0.028$) falls outside the range spanned by its own per-type correlations ($\rho \in [0.14, 0.50]$, Section 7.3) — a Simpson's-paradox effect that would mislead if the pooled figure were quoted on its own. We report and interpret only stratified figures for this reason.

8.3. Limitations and Future Work

1. **Safety and Security Integration:** SaG currently focuses on Reliability and Maintainability. Incorporating formal hazard categories (e.g., ISO 26262 ASIL ratings) and security threat models into the multigraph schema represents an important direction for future research.
2. **Hardware-in-the-Loop (HIL) Validation:** Validating SaG's predictions against real-time physical fault-injection testbeds in cyber-physical environments.
3. **Automated Architectural Refactoring:** Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies and add redundancy.

8.4. Conclusion

We presented **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that models complex distributed pub-sub and microservice architectures as typed, weighted multigraphs. By combining relation-specific Heterogeneous Graph Transformers with an interpretable Reliability–Maintainability quality attribution model, SaG bridges the Architecture–Code Gap, forecasting cascading failure risks before systems are deployed.

Our empirical results across synthetic and authentic open-source distributed systems demonstrate that heterogeneous graph learning provides significant advantages in identifying critical components ($F_1@K = 0.467$) and generalizing across unseen architectures ($\rho = 0.668$), while delivering actionable architectural diagnostics for resilient distributed systems engineering.

CRediT authorship contribution statement

[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Omitted for double-anonymised review.]

Data availability

The replication package—including the seven synthetic scenario datasets, topology generators, simulation harnesses, and real-world architecture adapters—will be made openly available upon publication.

Declaration of generative AI use

[To be completed by the authors in accordance with the journal's policy.]

References

[1] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[2] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[3] OASIS, MQTT version 5.0, OASIS Standard (2019).

[4] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[6] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[7] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[8] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.

[9] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.

[10] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.

[11] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.

[12] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.

[13] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.

[14] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.

[15] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.

[16] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.

[17] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.

[19] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.

[20] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.

[21] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.

[22] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.

[23] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.

[24] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.

[25] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.

[26] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.

[27] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.

[28] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.

[29] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.

[30] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.

[31] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000) 378–382.

[32] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).

[33] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.

[34] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.

[35] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.

[36] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.

[37] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.

[38] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.

[39] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.

[40] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.

[41] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).

[42] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).

[43] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.

[44] Team SimPy, Simpy: Event discrete simulation for Python, `https://simpy.readthedocs.io` (2020).

[45] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.

[46] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.

[47] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).

[48] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.

[49] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[50] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[51] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.